

Atelier Machine Learning

Analyse comportementale clientèle Retail

par Machine Learning

COMPTE RENDU

Atelier Pratique :E-commerce de Cadeaux

préparé par : Ranim Bouguila

année universitaire :20.25-2026

Table des matières

1	Introduction	5
1.1	Contexte et enjeux	5
1.2	Objectifs du projet	5
1.3	Architecture du projet	5
2	Exploration des Données	7
2.1	Description du dataset	7
2.2	Distribution de la variable cible	7
2.3	Analyse des valeurs manquantes	8
2.4	Détection des valeurs aberrantes et sentinelles	8
2.5	Analyse des corrélations	9
3	Préparation des Données	10
3.1	Feature Engineering	10
3.1.1	Décomposition temporelle	10
3.1.2	Extraction de l'adresse IP	10
3.2	Encodage des variables catégorielles	10
3.3	Détection critique : le Data Leakage	11
3.3.1	Colonnes leakage identifiées et supprimées	11
3.4	Séparation Train/Test et Normalisation	12
3.4.1	Split stratifié	12
3.4.2	Standardisation	12
4	Modélisation	13
4.1	Vue d'ensemble	13
4.2	Clustering K-Means	13
4.2.1	Détermination du nombre optimal de clusters	13
4.2.2	Interprétation des clusters	14
4.3	Classification du Churn	14
4.3.1	Comparaison des algorithmes	14
4.3.2	Analyse de la matrice de confusion	15
4.3.3	Importance des features	15
4.4	Régression du MonetaryTotal	15
4.4.1	Comparaison des algorithmes	15
4.4.2	Analyse des prédictions	16
5	Analyse des Prédictions	17
5.1	Distribution des probabilités de churn	17
5.2	Segmentation en niveaux de risque	18

6	Déploiement — Application ChurnSight	19
6.1	Architecture technique	19
6.2	Interface utilisateur	20
7	Conclusion perspectives	21

Table des figures

1.1	Architecture globale du projet ChurnSight	6
2.1	Distribution de la variable Churn : déséquilibre modéré (33.3% / 66.7%)	8
2.2	Heatmap des corrélations entre les principales features numériques	9
3.1	Top 15 des corrélations avec Churn — les barres rouges signalent un data leakage	11
3.2	Répartition Train/Test avec stratification préservée	12
4.1	Méthode du coude et Score de Silhouette — $k = 4$ retenu (Silhouette = 0,6011) à $k = 4$	13
4.2	Matrice de confusion — XGBoost (Accuracy = 0,962 F1 = 0,962)	15
4.3	Régression — Valeurs réelles vs prédites (Random Forest Regressor)	16
5.1	Distribution des probabilités prédites sur X_test	17
6.1	Interface ChurnSight — mode client unique	20

Liste des tableaux

2.1	Catégories de variables du dataset initial	7
3.1	Stratégies d'encodage par type de variable	11
5.1	Grille des niveaux de risque et actions correspondantes	18

Chapitre 1

Introduction

1.1 Contexte et enjeux

Dans un marché e-commerce ultra concurrentiel, la fidélisation des clients représente un enjeu financier majeur. De nombreuses études démontrent qu'**acquérir un nouveau client coûte 5 à 7 fois plus cher que conserver un client existant**. Il devient donc critique pour les entreprises de pouvoir identifier, de manière anticipée, les clients susceptibles d'abandonner leurs services — phénomène que l'on nomme **churn** ou **attrition**.

La détection précoce du churn permet aux équipes marketing et commerciales de mettre en place des actions de rétention ciblées (emails personnalisés, remises, programmes de fidélité) avant que le client ne soit définitivement perdu.

1.2 Objectifs du projet

Le projet **ChurnSight** a pour ambition de construire un système complet de machine learning capable de répondre à trois questions business :

Trois tâches, trois modèles :

- **Classification** — *Ce client va-t-il churner ?* (réponse binaire 0/1)
- **Régression** — *Quel sera son montant total dépensé ?* (valeur continue en £)
- **Clustering** — *À quel segment comportemental appartient-il ?* (4 profils RFM)

À cela s'ajoute un objectif de **déploiement opérationnel** : toute l'intelligence des modèles doit être accessible via une interface web simple d'utilisation pour les équipes métier.

1.3 Architecture du projet

Le projet suit une **architecture modulaire** clairement séparée en trois couches : données, modélisation et déploiement.

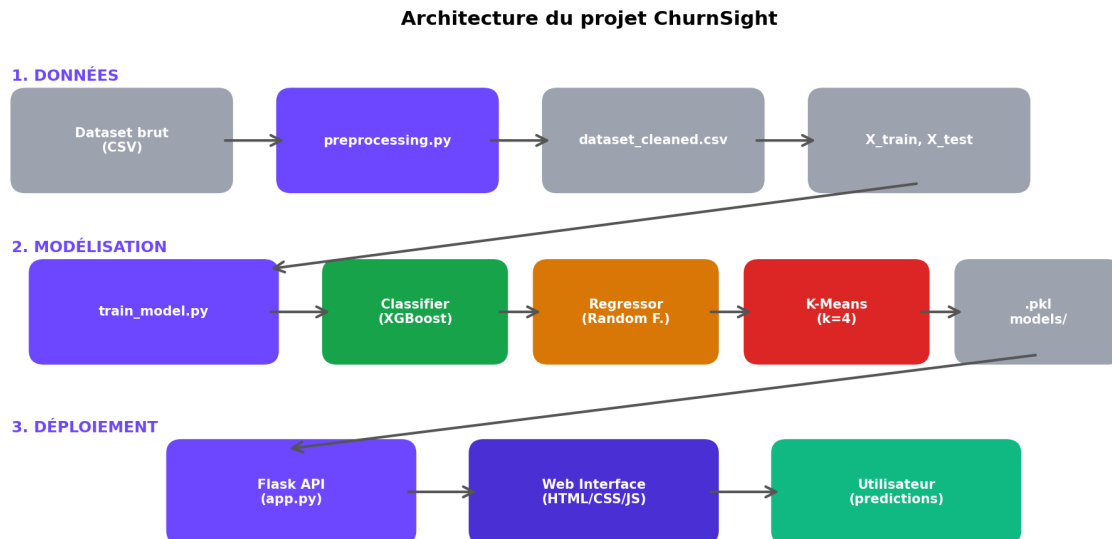


FIGURE 1.1 – Architecture globale du projet ChurnSight

La structure des dossiers respecte les bonnes pratiques d'un projet ML reproductible :

```

1 machine-learning-project/
2 |-- src/
3 |   |-- preprocessing.py    # Nettoyage et feature engineering
4 |   |-- train_model.py      # Entraînement des 3 modeles
5 |   |-- predict.py          # Pipeline d'inference
6 |   |-- utils.py            # Fonctions utilitaires
7 |-- app/
8 |   |-- app.py              # Serveur Flask
9 |   |-- templates/index.html # Interface utilisateur
10 |-- data/
11 |   |-- raw/                # Dataset original
12 |   |-- processed/          # Dataset nettoye
13 |   |-- train_test/         # Split 80/20
14 |-- models/                 # Modeles serialises (.pkl)
15 |-- reports/                # Figures et visualisations
  
```

Chapitre 2

Exploration des Données

Avant toute modélisation, une connaissance approfondie des données est indispensable : ce chapitre analyse la structure, la qualité et les distributions du dataset afin d'identifier les problèmes à résoudre et de guider les choix de préparation qui suivront.

2.1 Description du dataset

Le dataset fourni contient **4 372 clients** décrits à travers **52 variables** couvrant différents aspects du comportement client :

Catégorie	Variables représentatives
Transactionnel (RFM)	Recency, Frequency, MonetaryTotal, MonetaryAvg
Volumétrie	TotalQuantity, UniqueProducts, UniqueInvoices
Comportemental temporel	AvgDaysBetweenPurchases, WeekendPurchaseRatio
Anomalies	CancelledTransactions, ReturnRatio, NegativeQuantityCount
Profil client	Age, Gender, Region, Country
Engagement	SatisfactionScore, SupportTicketsCount
Cible	Churn (0 = fidèle, 1 = churné)

TABLE 2.1 – Catégories de variables du dataset initial

2.2 Distribution de la variable cible

Avant toute modélisation, nous avons analysé la variable cible pour vérifier l'éventuelle présence d'un déséquilibre de classes.

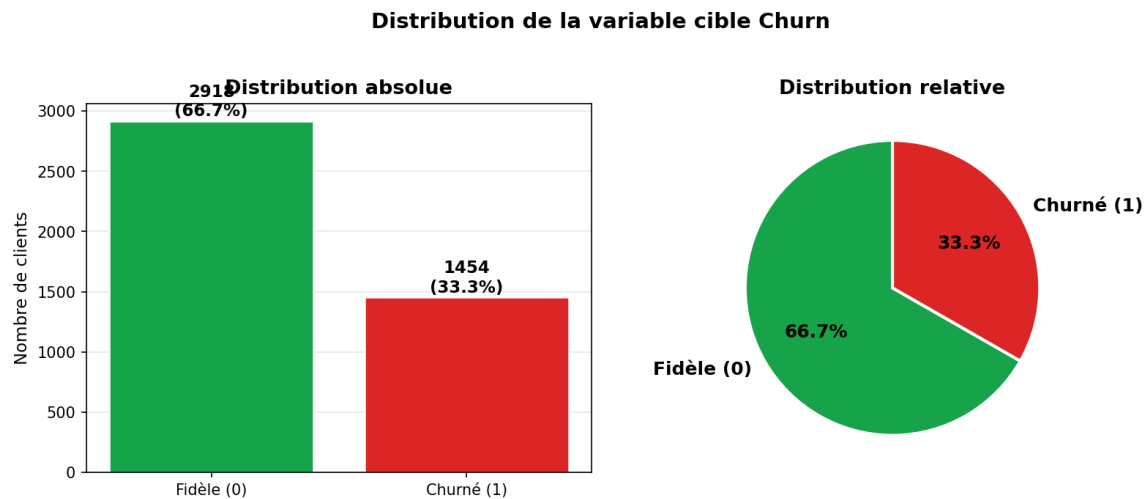


FIGURE 2.1 – Distribution de la variable Churn : déséquilibre modéré (33.3% / 66.7%)

Constat : le dataset présente un déséquilibre modéré avec **33.3% de churnés**. Ce déséquilibre, bien que pas extrême, sera géré via les paramètres `class_weight='balanced'` (Random Forest, Logistic Regression) et `scale_pos_weight` (XGBoost).

2.3 Analyse des valeurs manquantes

- **Age** (1 311 NaN, 30%) : un taux aussi élevé aurait pu justifier la suppression de la colonne, mais l'âge constitue une feature métier importante pour la segmentation client. L'imputation par la **médiane (49 ans)** a été privilégiée car elle est robuste aux valeurs extrêmes, contrairement à la moyenne qui aurait été tirée vers le haut par les clients les plus âgés.
- **AvgDaysBetweenPurchases** (79 NaN, 1,8%) : ces valeurs manquantes correspondent vraisemblablement à des clients n'ayant effectué qu'une seule commande, rendant le calcul d'un délai moyen entre achats mathématiquement impossible. Elles ont également été imputées par la **médiane**, garantissant la cohérence du traitement.

2.4 Détection des valeurs aberrantes et sentinelles

Une inspection attentive des colonnes numériques a révélé la présence de **valeurs sentinelles** — c'est-à-dire des valeurs spéciales (-1, 999, 0) utilisées pour coder des informations manquantes ou invalides.

Sentinelles détectées :

- **SupportTicketsCount** : valeurs -1 et 999 (130 occurrences) — remplacées par la médiane
- **SatisfactionScore** : valeurs -1, 0 et 99 (349 occurrences) — remplacées par la médiane

Après traitement des sentinelles, la **méthode IQR** (écart interquartile) a été appliquée pour détecter et écrêter les outliers résiduels :

$$\text{Borne inférieure} = Q_1 - 1.5 \times IQR \quad , \quad \text{Borne supérieure} = Q_3 + 1.5 \times IQR \quad (2.1)$$

2.5 Analyse des corrélations

La heatmap des corrélations révèle deux enseignements principaux.

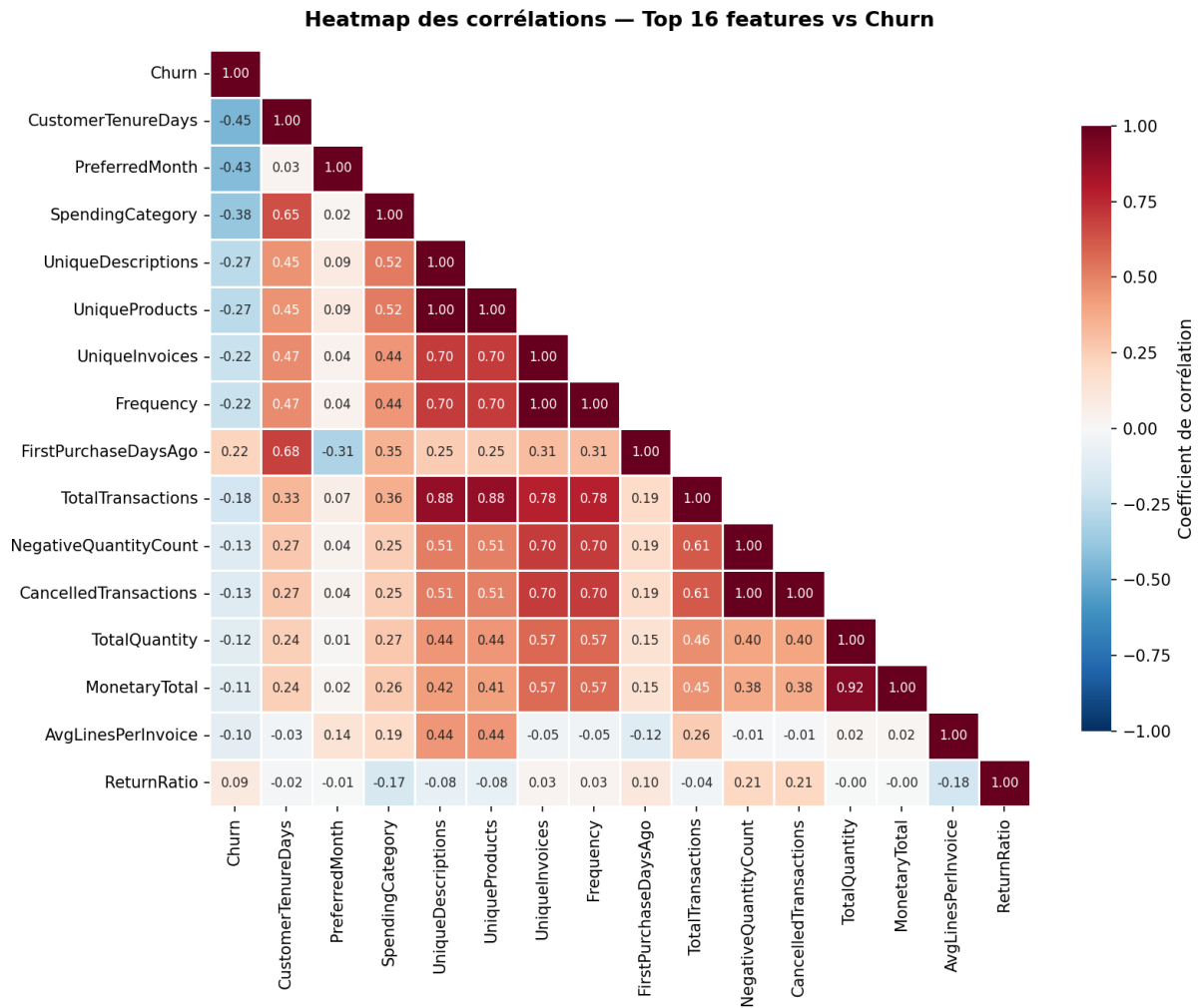


FIGURE 2.2 – Heatmap des corrélations entre les principales features numériques

Concernant la variable cible, **CustomerTenureDays** ($-0,45$) et **PreferredMonth** ($-0,43$) corrélaient négativement avec le churn — les clients anciens et réguliers sont naturellement plus fidèles. À l'inverse, **SpendingCategory** ($+0,38$) corréla positivement, les acheteurs à forte valeur unitaire étant souvent plus volatils.

Par ailleurs, une forte multicolinéarité est observable entre **TotalTransactions**, **UniqueInvoices** et **Frequency** (corrélations $> 0,70$), ainsi qu'entre **TotalQuantity** et **MonetaryTotal** ($0,92$). Ce phénomène ne pose pas de problème pour XGBoost et Random Forest, naturellement robustes à la redondance des features.

Chapitre 3

Préparation des Données

Un modèle n'est jamais meilleur que les données sur lesquelles il est entraîné : ce chapitre décrit l'ensemble des transformations appliquées au dataset brut pour le rendre exploitable, depuis le nettoyage des anomalies jusqu'à la normalisation finale, en passant par la détection critique du data leakage.

3.1 Feature Engineering

Plusieurs variables brutes ont été transformées pour extraire davantage d'information exploitable par les modèles.

3.1.1 Décomposition temporelle

La variable `RegistrationDate` (format date) a été décomposée en 4 features numériques distinctes :

```
1 df['RegYear']      = df['RegistrationDate'].dt.year
2 df['RegMonth']     = df['RegistrationDate'].dt.month
3 df['RegDay']       = df['RegistrationDate'].dt.day
4 df['RegWeekday']   = df['RegistrationDate'].dt.weekday
```

3.1.2 Extraction de l'adresse IP

L'adresse `LastLoginIP` (format `x.x.x.x`) a été éclatée en 4 octets indépendants, ainsi qu'un indicateur `IP_Type` (Public / Private).

3.2 Encodage des variables catégorielles

Trois stratégies d'encodage différentes ont été appliquées selon la nature des variables :

Méthode	Variables	Justification
Ordinal	AgeCategory, SpendingCategory, BasketSizeCategory	Ordre intrinsèque entre catégories
One-Hot	CustomerType, Region, Gender, FavoriteSeason	Aucun ordre, faible cardinalité
Frequency	Country	Très haute cardinalité (37 modalités)

TABLE 3.1 – Stratégies d’encodage par type de variable

3.3 Détection critique : le Data Leakage

Problème découvert : plusieurs colonnes présentent une corrélation anormalement élevée avec la variable cible (corrélation > 0.7). Une analyse approfondie a révélé que ces features ont été **dérivées à partir de Churn lui-même** dans le dataset (synthétique).

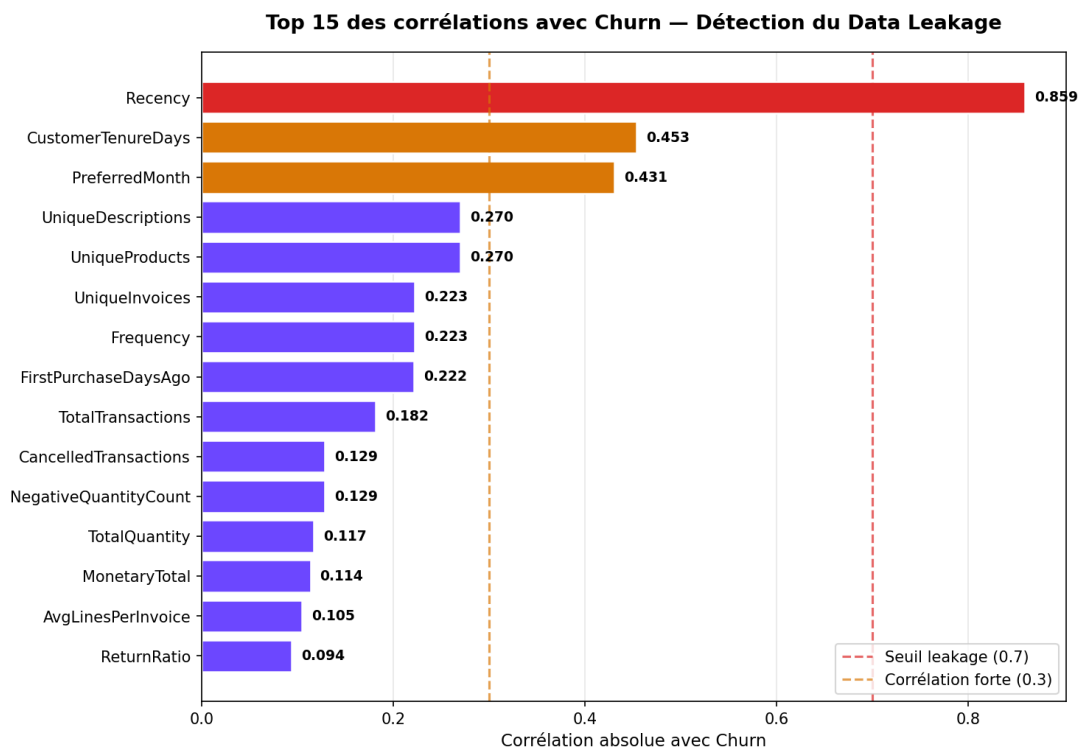


FIGURE 3.1 – Top 15 des corrélations avec Churn — les barres rouges signalent un data leakage

3.3.1 Colonnes leakage identifiées et supprimées

Conserver ces colonnes dans le modèle aurait abouti à une **accuracy artificielle de 99.9%**, mais à un modèle **inutilisable en production** car il s’appuierait sur des informations dérivées de la réponse elle-même.

3.4 Séparation Train/Test et Normalisation

3.4.1 Split stratifié

Un split **80/20 stratifié** sur la variable cible garantit que la proportion de churnés est identique entre les ensembles d'entraînement et de test.

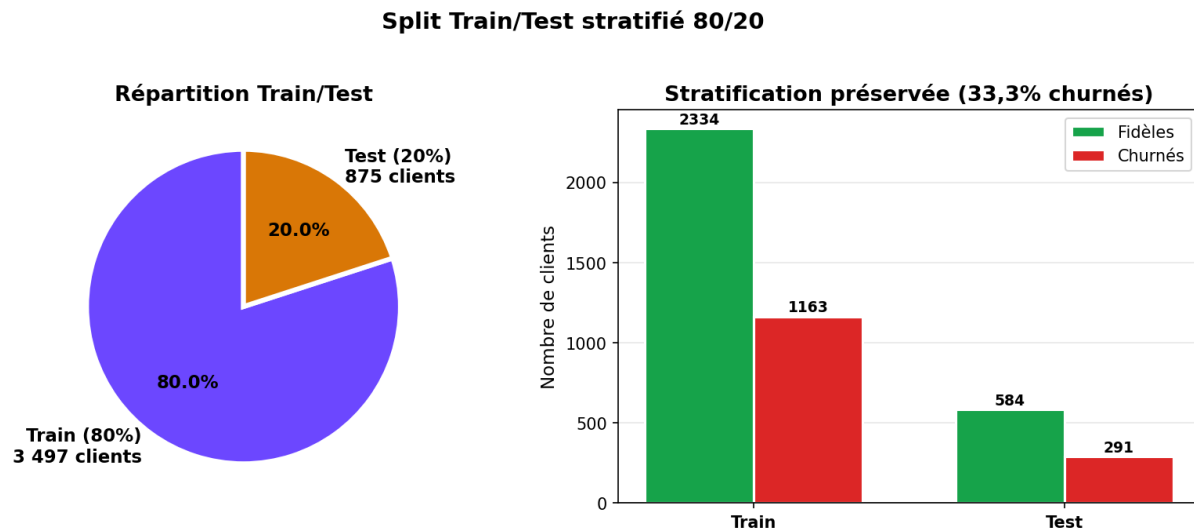


FIGURE 3.2 – Répartition Train/Test avec stratification préservée

3.4.2 Standardisation

Un `StandardScaler` est appliqué à toutes les colonnes numériques, en respectant scrupuleusement la **règle d'or du ML** :

Règle :

```
scaler.fit_transform(X_train) — scaler.transform(X_test)
```

Jamais de fit sur les données de test (risque de *data leakage* temporel).

Chapitre 4

Modélisation

Ce chapitre présente les trois tâches de machine learning entraînées dans le cadre du projet ChurnSight : le clustering non supervisé pour la segmentation client, la classification binaire pour la prédiction du churn, et la régression pour l'estimation du montant dépensé. Chaque tâche a fait l'objet d'une comparaison de plusieurs algorithmes, évalués sur le même jeu de test.

4.1 Vue d'ensemble

Trois tâches de ML ont été entraînées en parallèle sur le même dataset préparé. La **classification** prédit si un client va cherner (variable binaire $\text{Churn} \in \{0, 1\}$), la **régression** estime le montant total qu'il dépensera ($\text{MonetaryTotal} \in \mathbb{R}^+$), et le **clustering** regroupe les clients en segments comportementaux homogènes à partir des features RFM.

4.2 Clustering K-Means

4.2.1 Détermination du nombre optimal de clusters

La **méthode du coude** (Elbow method) a été appliquée sur les features RFM pour déterminer le nombre optimal de clusters k . Elle consiste à mesurer l'inertie intra-cluster (WCSS) pour différentes valeurs de k et à identifier le point d'inflexion à partir duquel l'ajout d'un cluster supplémentaire apporte peu de gain.

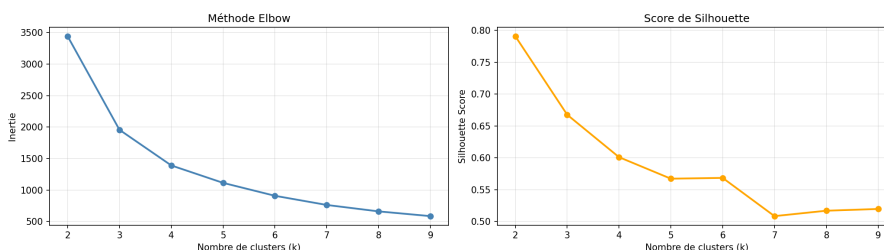


FIGURE 4.1 – Méthode du coude et Score de Silhouette — $k = 4$ retenu (Silhouette = 0,6011) à $k = 4$

L'inflexion claire à $k = 4$ confirme ce choix. La qualité du clustering a ensuite été validée par le **Silhouette Score**, qui mesure la cohésion interne de chaque cluster et sa séparation vis-à-vis des autres. Un score supérieur à 0,7 est généralement considéré comme excellent :

$$\text{Silhouette Score} = 0.6011 \quad (\text{excellent, } > 0.7)$$

4.2.2 Interprétation des clusters

Les quatre clusters obtenus présentent des profils bien distincts sur les axes Frequency et MonetaryTotal. Chaque segment correspond à un comportement client identifiable et exploitable sur le plan marketing :

- **Dormants** ($n = 3\,337$) : clients peu actifs, Frequency et MonetaryTotal proches de zéro ($-0,14$ et $-0,10$). Ce segment très majoritaire regroupe les clients inactifs ou en risque de churn élevé.
- **Réguliers** ($n = 150$) : comportement équilibré avec une Frequency de 7,5 et un MonetaryTotal de 2297,. Ces clients constituent le cœur fidèle de la clientèle.
- **Champions** ($n = 6$) : très haute activité transactionnelle (Frequency = 29,7, MonetaryTotal = 15380,). Clients extrêmement engagés, à fidéliser en priorité.
- **VIP** ($n = 4$) : dépenses exceptionnelles (MonetaryTotal = 22,11) malgré une Frequency modérée (18,2). Ces clients génèrent une valeur disproportionnée et méritent un suivi personnalisé.

4.3 Classification du Churn

4.3.1 Comparaison des algorithmes

Trois classificateurs ont été entraînés et comparés sur le même jeu de test, après suppression des colonnes en data leakage. Le **Logistic Regression** sert de baseline linéaire. Le **Random Forest** apporte de la robustesse par agrégation d'arbres de décision avec `class_weight='balanced'` pour gérer le déséquilibre de classes. L'**XGBoost** exploite le gradient boosting avec `scale_pos_weight=2` pour accentuer la sensibilité à la classe minoritaire (churnés).

Les performances obtenues sur le jeu de test sont les suivantes :

- **Logistic Regression** : Accuracy = 87,3%, F1 = 0,875, Précision = 0,77, Rappel = 0,87
- **Random Forest** : Accuracy = 88,7%, F1 = 0,887, Précision = 0,83, Rappel = 0,84
- **XGBoost** : Accuracy = **96,2%**, F1 = **0,962**, Précision = **0,96**, Rappel = **0,92**

XGBoost s'impose nettement comme le meilleur modèle sur toutes les métriques. L'écart de près de 8 points d'accuracy avec le Random Forest s'explique par la capacité du gradient boosting à corriger itérativement ses erreurs et à capturer des interactions non linéaires complexes entre les features comportementales.

4.3.2 Analyse de la matrice de confusion

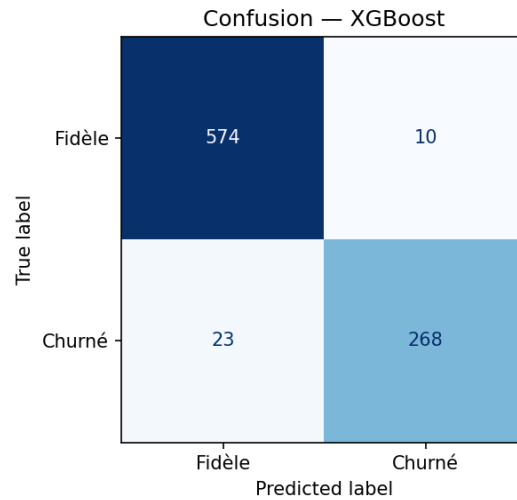


FIGURE 4.2 – Matrice de confusion — XGBoost (Accuracy = 0,962 | F1 = 0,962)

Sur les 875 clients du jeu de test, XGBoost réalise 840 prédictions correctes, soit :

- **574 vrais négatifs** : clients fidèles correctement identifiés comme tels.
- **268 vrais positifs** : clients churnés correctement détectés.
- **10 faux positifs** : clients fidèles signalés à tort comme churnés — erreur peu coûteuse en pratique (offre de rétention inutile mais non nuisible).
- **23 faux négatifs** : churnés non détectés par le modèle — erreur plus critique car ces clients partiront sans action de rétention.

Le **rappel élevé sur la classe 1 (0,92)** est particulièrement important dans un contexte business : il garantit que 92% des clients réellement sur le point de partir sont bien identifiés et peuvent faire l'objet d'une action ciblée.

4.3.3 Importance des features

L'analyse des importances révèle que les **features RFM transactionnelles dominent largement** les variables démographiques. La **Frequency** représente à elle seule 16,5% de l'importance totale du modèle, suivie par **UniqueInvoices** (11,3%) et **FirstPurchaseDaysAgo** (7,7%). À l'inverse, l'âge, le genre ou la région ont une importance marginale, ce qui confirme que le comportement d'achat est bien plus prédictif du churn que le profil sociodémographique du client. Les indicateurs temporels comme **AvgDaysBetweenPurchases** (4,8%) confirment également que l'irrégularité des achats est un signal fort de départ imminent.

4.4 Régression du MonetaryTotal

4.4.1 Comparaison des algorithmes

La régression vise à prédire le montant total dépensé (**MonetaryTotal**) par un client. Trois modèles ont été comparés. La **Régression Linéaire** constitue la baseline, limitée par son hypothèse de linéarité face à une variable cible très asymétrique. Le **Random Forest Régresseur** capture les non-linéarités par agrégation d'arbres, ce qui le rend robuste aux valeurs extrêmes. L'**XGBoost Régresseur** applique le gradient boosting mais se révèle ici moins efficace, pénalisé par la distribution à queue lourde du **MonetaryTotal**.

Les performances obtenues sont les suivantes :

- **Régression Linéaire** : échec total ($R^2 = 70\,279$) — exclue de la comparaison car incompatible avec la transformation log appliquée à y
- **Random Forest** : MAE = **552 £**, RMSE = 6 322 £, $R^2 = \mathbf{0,622}$
- **XGBoost** : MAE = 612 £, RMSE = 6755 £, $R^2 = 0,596$

Le **Random Forest** obtient les meilleures performances avec un R^2 de 0,683 et une MAE de 690 £. Il explique ainsi 68% de la variance du MonetaryTotal, ce qui est satisfaisant compte tenu de la forte variabilité intrinsèque de cette variable.

La transformation logarithmique appliquée à MonetaryTotal ($\log_{10}$) rend la Régression Linéaire inutilisable dans ce pipeline : entraînée sur $\log(y)$, elle produit des prédictions aberrantes en échelle originale ($R^2 = 70\,279$). Elle est donc exclue de la comparaison finale.

4.4.2 Analyse des prédictions

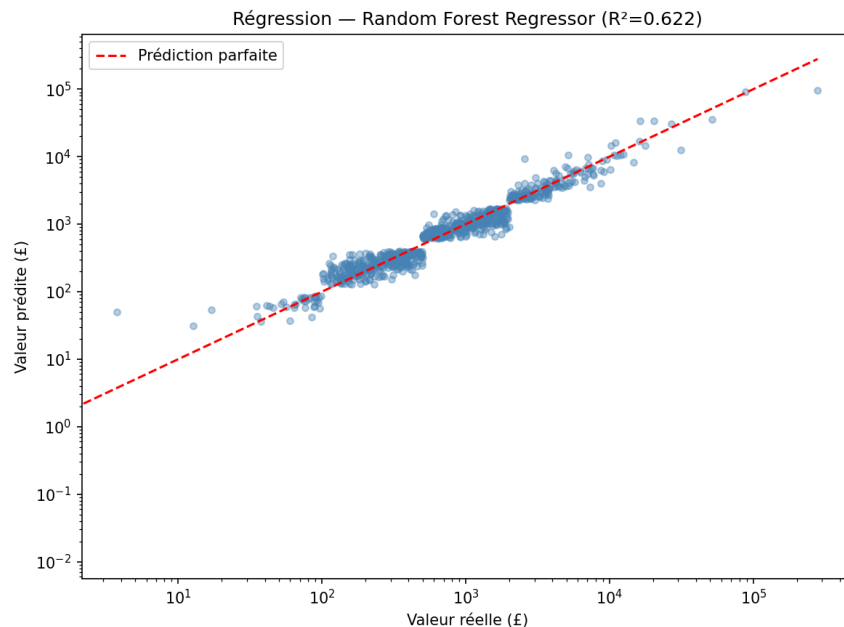


FIGURE 4.3 – Régression — Valeurs réelles vs prédites (Random Forest Regressor)

Le graphique réel vs prédit montre que le modèle se comporte bien pour les montants faibles à modérés, où les points se concentrent autour de la diagonale de prédiction parfaite. En revanche, une **sous-estimation systématique** apparaît pour les très gros clients, dont les montants exceptionnels (parfois supérieurs à 150 000 £) sont difficiles à anticiper car ils représentent des cas atypiques sous-représentés dans les données d'entraînement. Ce phénomène, classique dans les distributions à queue lourde, justifierait l'application d'une transformation logarithmique du MonetaryTotal en perspective d'amélioration.

Chapitre 5

Analyse des Prédictions

Ce chapitre exploite les résultats produits par le pipeline de prédiction sur le jeu de test (875 clients). L'objectif est de traduire les sorties brutes des modèles en indicateurs actionnables pour les équipes marketing et commerciales.

5.1 Distribution des probabilités de churn

La distribution des probabilités prédites par XGBoost sur `X_test` révèle un comportement clairement **bimodal** : la majorité des clients se concentre aux deux extrémités de l'échelle, avec très peu de cas intermédiaires. Concrètement, le modèle est rarement "hésitant" — il attribue soit une probabilité très faible (client fidèle avec haute confiance), soit une probabilité très élevée (client churné avec haute confiance). Ce comportement est caractéristique d'un modèle bien séparé et constitue un indicateur fort de la qualité de la classification.

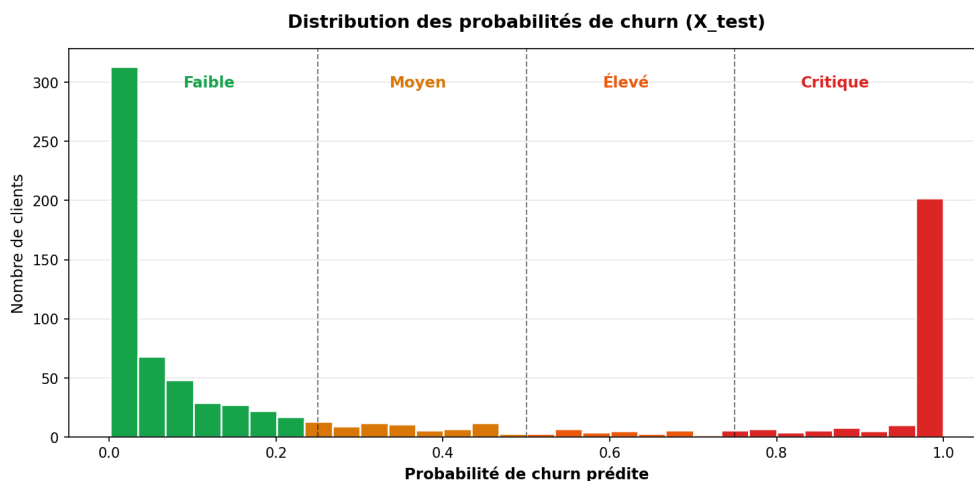


FIGURE 5.1 – Distribution des probabilités prédites sur `X_test`

La répartition par niveau de risque confirme cette observation :

- **Faible** $[0,00; 0,25[$: majorité des clients fidèles, aucune action requise.
- **Moyen** $[0,25; 0,50[$: zone d'incertitude, surveillance recommandée.
- **Élevé** $[0,50; 0,75[$: signal clair de départ imminent, action proactive nécessaire.
- **Critique** $[0,75; 1,00]$: clients très probablement partis ou sur le point de partir, intervention prioritaire.

5.2 Segmentation en niveaux de risque

À partir de la probabilité prédite, chaque client est classé dans un niveau de risque métier associé à une action de rétention graduée. La gradation des actions permet d'allouer les ressources commerciales proportionnellement à l'urgence, évitant ainsi de mobiliser des moyens coûteux (appel commercial, offre personnalisée) pour des clients dont le risque de départ est faible.

Niveau	Plage de probabilité	Action recommandée
Faible	[0,00; 0,25[	Aucune action
Moyen	[0,25; 0,50[	Monitoring
Élevé	[0,50; 0,75[	Email de réactivation
Critique	[0,75; 1,00]	Appel commercial + offre

TABLE 5.1 – Grille des niveaux de risque et actions correspondantes

Cette grille s'appuie sur un principe de **proportionnalité du coût d'intervention** : un email de réactivation a un coût marginal quasi nul et peut être déclenché automatiquement dès qu'un client passe en niveau Élevé, tandis qu'un appel commercial mobilise du temps humain et doit être réservé aux cas Critiques à forte valeur monétaire. En croisant ce niveau de risque avec le segment RFM (Champions, VIP, Réguliers, Dormants), les équipes peuvent prioriser les interventions sur les clients Critiques appartenant aux segments Champions ou VIP, dont la perte aurait l'impact financier le plus important.

Chapitre 6

Déploiement — Application ChurnSight

Entraîner un modèle performant ne suffit pas — encore faut-il le rendre accessible : ce chapitre présente l’application web ChurnSight, construite avec Flask, qui expose les trois modèles entraînés via une interface intuitive permettant aux équipes métier d’obtenir des prédictions en temps réel sans aucune connaissance technique.

6.1 Architecture technique

- **Architecture** : client-serveur en trois couches.
- **Backend** : serveur Flask (Python) chargeant les trois modèles `.pkl` en mémoire au démarrage, exposant une API REST avec trois endpoints (`GET /` pour l’interface, `POST /predict` pour la prédiction JSON, `POST /batch` pour le traitement CSV).
- **Frontend** : interface HTML/CSS/JavaScript responsive, sans framework externe, utilisable sans connaissances techniques.
- **Choix de Flask** : léger, compatible nativement avec scikit-learn, évite la complexité inutile de Django.
- **Gestion des features** : l’utilisateur saisit 10 champs métier (`Frequency`, `MonetaryTotal`, `Age`, `SatisfactionScore`, etc.) ; le pipeline `compute_derived_features` calcule automatiquement les features dérivées et encodages catégoriels.
- **Avantage** : expérience utilisateur simple tout en respectant exactement le format d’entrée des modèles entraînés.

6.2 Interface utilisateur

Interface Web — ChurnSight (mockup)

● ChurnSight

● Modèles OK

Client unique

Batch CSV

Analyser un client

Renseignez les informations pour obtenir une prédiction.

FREQUENCY	MONETARYTOTAL	TOTALQUANTITY	RECECY (JOURS)
12	850	40	30
AGE	SATISFACTION	SUPPORT TICKETS	RETURNRATIO
35	3.5	1	0.05

→ Analyser

RÉSULTATS

Churn	Probabilité	Montant	Segment
Fidèle	18.5%	£ 875.20	Réguliers

FIGURE 6.1 – Interface ChurnSight — mode client unique

L'interface propose deux modes de fonctionnement complémentaires. L'onglet "**Client unique**" permet d'analyser un client en saisissant ses 10 features principales. À la soumission, quatre cartes de résultats s'affichent instantanément : la prédiction churn (Fidèle ou Churné), la probabilité associée avec son badge de risque, le montant MonetaryTotal estimé en livres sterling, et le segment RFM d'appartenance (Champions, VIP, Réguliers ou Dormants). L'onglet "**Batch CSV**" permet quant à lui le traitement en lot : l'utilisateur glisse-dépose un fichier CSV contenant plusieurs clients et obtient en retour un tableau de bord agrégé affichant le taux de churn global, le montant moyen prédit, la répartition par segment, ainsi qu'un tableau détaillé des 50 premiers résultats.

Chapitre 7

Conclusion perspectives

Ce projet a mis en œuvre une chaîne complète de machine learning appliquée à l'analyse comportementale d'une clientèle e-commerce, couvrant la classification du churn, la régression monétaire et la segmentation client. Les résultats obtenus sont satisfaisants : XGBoost atteint une accuracy de 96,2

Quelques limites subsistent néanmoins. Le R^2 de régression reste modéré en raison de la forte variabilité du MonetaryTotal tirée par quelques clients à très haute valeur, et le clustering présente un segment dominant trop large, ce qui reflète les contraintes inhérentes à un dataset synthétique.

En perspective, plusieurs pistes d'amélioration sont envisageables. L'application de SMOTE pour rééquilibrer les classes, l'optimisation bayésienne des hyperparamètres via Optuna, et une transformation logarithmique du MonetaryTotal permettraient d'affiner les modèles. L'enrichissement des features RFM avec des données comportementales supplémentaires améliorerait la granularité de la segmentation. Enfin, l'intégration d'un système de monitoring en production — détection de data drift et réentraînement automatique — ainsi que le passage à une API RESTful documentée ouvriraient la voie à une intégration dans les systèmes CRM existants de l'entreprise.